Cafeteria Order Management System

A Major Project Report

Submitted to Mr . Nabeel Jarral

Bachelor of sciences

in

Artificial Intelligence

by

Minahil Mazhar (05)

Hadia Naseer(38)

2023-UOK-04805

2023-UOK-04838

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

UNIVERSITY OF KOTLI

Kotli,AJK

October 2024

Abstract

This report provides a detailed overview of the project's development, focusing on system architecture and implementation. It explains the methodologies used to solve technical challenges and presents the final results and conclusions. To help with understanding, the document includes practical examples of coding structures, data visualization, and academic referencing. This guide serves as a complete technical resource for utilizing LaTeX in reporting.

Contents

List of Figures

Chapter 1

Introduction

Our project focuses on developing a ”Cafeteria Order Management System,” designed with simplicity in mind for beginners. This system is specifically tailored for students, allowing them to conveniently order food from nearby shops without disrupting their studies.

In today’s fast-paced, technology-driven world, people increasingly prefer online solutions over traditional purchasing methods. Recognizing this trend, our system leverages artificial intelligence to streamline the ordering process. We understand that stepping out to order food—whether it’s a burger or a cup of coffee—can be time-consuming and inconvenient, especially after spending long hours in extreme temperatures.

The Cafeteria Order Management System is designed to save time and maintain accurate records, enabling users to reorder their favorite meals effortlessly. One of its standout features is the notification system, which alerts users when their food is ready for pickup.

Customer satisfaction and user-friendliness are at the core of our design. The system is intuitive, ensuring that even non-technical users can easily navigate its features. Should users encounter any issues, they can provide feedback seamlessly, which the system values and uses to enhance its functionality.

In summary, our project aims to provide a practical solution that caters to the needs of students while prioritizing efficiency and user satisfaction.

This system leverages technology to provide a seamless online ordering experience, enabling students to place food orders from nearby cafeterias at their convenience. By utilizing Object-Oriented Programming (OOP) principles, our project aims to create

a modular, maintainable, and scalable application that enhances user experience and operational efficiency.

OOP is a programming paradigm that emphasizes the use of ”objects” to represent real-world entities, allowing for better organization of code and improved collaboration among developers. Key OOP concepts such as encapsulation, inheritance, and polymorphism will be employed to design the system's architecture.

Encapsulation will allow us to bundle data and methods that operate on that data within classes, ensuring a clear separation of concerns and enhancing security.
Inheritance will enable us to create a hierarchy of classes, allowing for code reusability and the extension of existing functionalities without modifying the original code.
Polymorphism will allow our system to process objects differently based on their data type or class, enhancing flexibility and adaptability.
The Cafeteria Order Management System will not only streamline the ordering process but also provide features such as order tracking, user feedback, and a recommendation system based on user preferences. By integrating these functionalities, we aim to create a user-friendly platform that enhances customer satisfaction while supporting local businesses.

In summary, this project seeks to combine the principles of OOP with the practical needs of students, resulting in an efficient and effective solution for food ordering in an academic setting.

1.1 Objective

The objective of the Cafeteria Order Management System is to develop an efficient, user-friendly platform that enables students to conveniently order food from nearby cafeterias and restaurants. By leveraging Object-Oriented Programming principles, the system aims to:

Streamline the Ordering Process: Provide a seamless online interface for students to place, modify, and track their food orders at any time, thereby minimizing time spent away from their studies.

Enhance User Experience: Implement features such as order notifications and personalized recommendations based on user preferences to improve overall satisfaction.

Ensure Scalability and Maintainability: Apply Object-Oriented Programming techniques to ensure the system is modular, easily maintainable, and scalable for future enhancements.

1.2 Scope

The scope of the Cafeteria Order Management System encompasses the functionalities, features, and limitations of the project. It defines what the system will and will not cover, ensuring clarity for stakeholders and guiding the development process. The scope includes the following aspects:

1. **Functional Scope**

   Menu Management: Allow cafeteria and restaurant administrators to update menus, including item descriptions, prices, and availability in real-time.

   Order Placement: Provide a user-friendly interface for students to browse menus and place orders, including options for customization (e.g.,add-ons).

   Notifications: Send notifications to users regarding order confirmation, preparation status, and when orders are ready for pickup.

   Admin : Admin can add items place and read orders and create their histories

2. **Technical Scope** Technology Stack: Utilize Object-Oriented Programming principles for the development of the application

   User Interface Design: Design a responsive and intuitive user interface that caters to a diverse user base, ensuring accessibility and ease of use.

3. **Limitations** Since it is designed with beginner mind so it is simple project . Setorder functions are not implemented on this project.

   Online payment: There is no procedure for online payment

1.3 Organization

First we have introduction includes why we designed this system.Then we have scope that describes functionality and constraints of the system.Objective of this project is the aim of this system. Now we have Literature review . I have read all the articles that describes work of different order management systems according to the authors point of view.At last we have UML diagram of cafeteria order management system .This is actually blue prints that describes all the aspects that will be used in system

Chapter 2

Literature Review

2.1 Review

I am going to give reviews of some research papers to show you how authors painted picture of order management system

2.1.1 A Design and Development of Part Management System including Capabilities from Data Management to Order Management

Rhee young "A Design and Development of Part Management System including Capabilities from Data Management to Order Management": Service Parts Management is defined as a supply management associated with service parts from the part suppliers to the final customer. A series of process to improve the customer service level by forecasting the demand and to minimize cost by maintaining the inventory level is included. Uniqueness such as missing value correction, the data pattern analysis and planned order system is designed and implemented. Main feature of order management system is to calculate order amount and order time based on selection of optimal forecasting algorithm.

2.1.2 A Study on the Efficiency of Cafeteria Management Systems

Shin Hyeong Choi"A Study on the Efficiency of Cafeteria Management Systems": Due to the high inflation rate of dining out, along with changes in group meals or cafeteria services, office workers are increasingly using workplace cafeterias to reduce their meal expenses even slightly. With the recent development of ICT technology, various fields are realizing that not only are smartphones becoming more popular, but they are also becoming an integration of the latest technologies. In this paper, we analyze the current status of cafeterias with a large number of customers and propose ways to improve problems or difficulties. Since most people always carry their smartphones for urgent communication or work tasks, we aim to develop a cafeteria management system that utilizes the NFC function of smartphones. By presenting the process from customer entry to menu selection, it will enable more efficient use of the cafeteria.

2.1.3 Investigating the integration of artificial intelligence in enhncing efficiency of distributed order management systems within sap environments

Sumen Shekhar"INVESTIGATING THE INTEGRATION OF ARTIFICIAL INTELLIGENCE IN ENHANCING EFFICIENCY OF DISTRIBUTED ORDER MANAGEMENT SYSTEMS WITHIN SAP ENVIRONMENTS":This research paper explores the integration of Artificial Intelligence (AI) into Distributed Order Management (DOM) systems within SAP environments, highlighting its role in enhancing efficiency and performance in decentralized supply chain operations. As supply chains become more complex, the adoption of advanced technologies is crucial for streamlining processes. The paper discusses how AI capabilities, including machine learning, predictive analytics, and automation, can improve order accuracy, reduce processing times, and adapt to changing market demands.

The study analyzes the technical foundations of AI and its application across various aspects of DOM systems. It assesses the current state of DOM systems in SAP environments, identifies relevant AI technologies, and evaluates the impact of AI integration on system efficiency. Additionally, the research addresses challenges related to AI implementation and proposes best practices for successful integration

in SAP-driven DOM settings. Ultimately, the paper aims to provide insights for organizations looking to leverage AI to enhance their DOM capabilities and gain a competitive advantage in the fast-evolving global supply chain landscape.

Chapter 3

Methodology

3.1 Classes

It is an OOP project that It contains three classes : admin ,staff and one abstract class MenuItemBase . Each class has its own specific functions that works accordingly .

3.2 Functions of MenuItemBase

getitem() It will get item .Actually it implements inheritance and polymorphism because of information hiding **getprice()** It will get price of an item . **getitemno()** It will get item number of item.

3.3 Functions of class staff

getitem() This function is inherited from abstract class ”MenuItemBase.”
getitemno() This function will get item number .This is under usage of staff. **getprice()**This function will get price of an item we already discussed.

3.4 For class admin

add() This function adds the item into the list.

order() It contains loops and conditional statements.The loop iterates over a collection of menuItems, . For each item, it retrieves:

getItemNo(): The item's number (or ID).

getItem(): The name or description of the item.

getPrice(): The price of the item.

It then prints this information in a formatted manner, displaying the item number, name, and price.

After this user will enter his choice . The conditional statement will checkout valid attempts and at the end it will calculate bill by multiplying price and quantity of item and show total bill **notify()** admin will notify customer,in how much time his food will be prepared .

3.5 Conclusion

In conclusion, the Cafeteria Order Management System represents a significant advancement in the way cafeterias manage their operations. By leveraging technology to streamline processes, enhance customer satisfaction, and provide actionable insights, the COMS positions cafeterias for success in a competitive landscape. As consumer expectations continue to evolve, adopting such systems will be essential for maintaining relevance and achieving operational excellence in the food service industry.

Chapter 4

Results

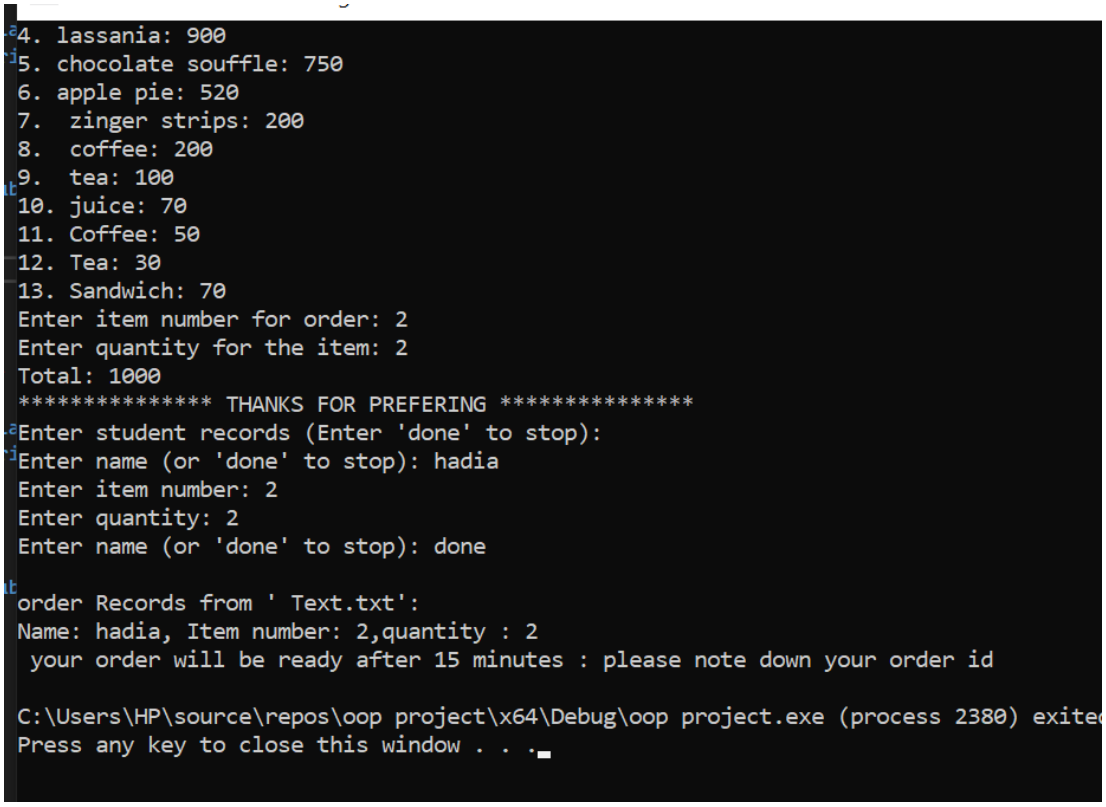

Figure 4.1: Output

First of all order function will display all the items due to which user can choose item. He will enter item number and quantity . That order function will calculate bill according to quantity and price and it will create order history . he can view his order history as well

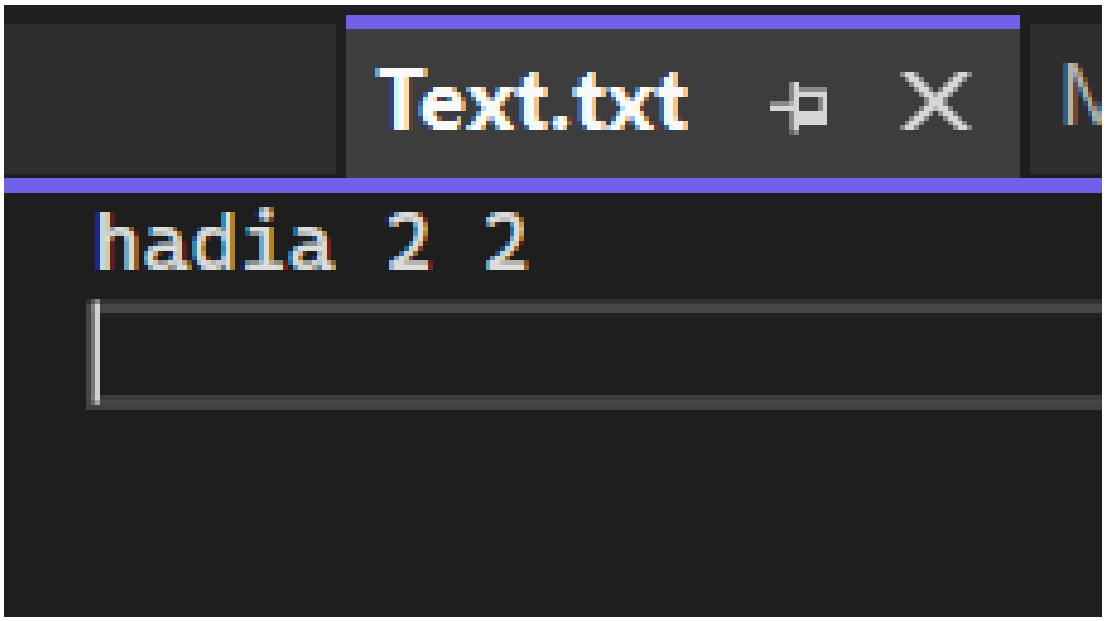

Figure 4.2: Text.txt file

Chapter 5

Appendix

5.1 System design

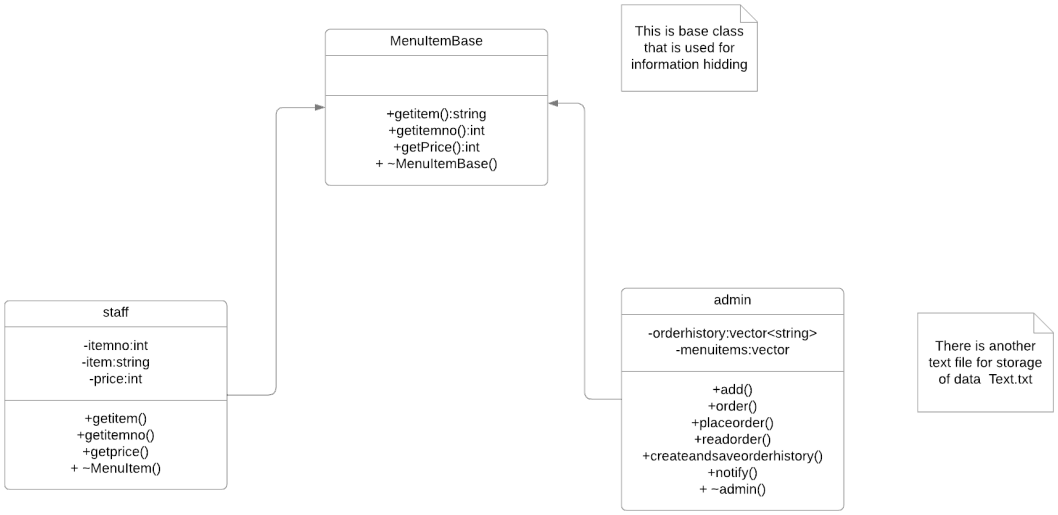

Figure 5.1: UML diagram

5.2 Code

5.2.1 MenuItemBase.h File

```
    #include <string>
#include <vector>
#include <iostream>
using namespace std;
```

```
class MenuItemBase {   //this is an abstract class
public:
    virtual string getItem() const = 0;
    virtual int getItemNo() const = 0;
    virtual int getPrice() const = 0;

    virtual ~MenuItemBase() {}
};
class admin {
private:

    vector<string> orderHistory;

    vector<MenuItemBase*> menuItems;
public:
    void add(int ITEMNO, string ITEM, int PRICE);
    void order();
    void placeOrder(const string& order);
    void readOrderHistory(const string& obj);
    void  createAndSaveOrderHistory(const string& obj, const vector <string>& o
    void notify();
    ~admin();
};

class staff : public MenuItemBase {   //INHERITANCE
private:
    int itemno;
    string item;
    int price;

public:
    staff(int ITEMNO, string ITEM, int PRICE); //constructor
    string getItem() const override; // Override
```

```
    int getItemNo() const override; // Override

    int getPrice() const override; // Override

    ~staff() {}
};
```

5.2.2 cafeteriaordermanagementsystem.cpp File

```
    #include "MenuItemBase.h"
#include<vector>
#include<fstream>
using namespace std;
staff::staff(int ITEMNO, string ITEM, int PRICE) { // constructor overloaded
    itemno = ITEMNO;
    item = ITEM;
    price = PRICE;  // resolution operator enables us to access function, class
}

string staff::getItem() const {   //encapsulation
    return item;
}

int staff::getItemNo() const {
    return itemno;
}

int staff::getPrice() const {
    return price;
}
void admin::add(int ITEMNO, string ITEM, int PRICE) {
```

```
    // usage of vector
    staff* menu = new staff(ITEMNO, ITEM, PRICE); // Dynamically allocate MenuI
    menuItems.push_back(menu);

}

void admin::order() {
    for (size_t i = 0; i < menuItems.size(); i++) {
        cout << menuItems[i]->getItemNo() << ". " << menuItems[i]->getItem()
            << ": " << menuItems[i]->getPrice() << endl;
    }
    //displaying items
    int ord, quantity; // this is for user
    cout << "Enter item number for order: ";
    cin >> ord;
    cout << "Enter quantity for the item: ";
    cin >> quantity;
    if (ord >= 14) {
        cout << "invalid option ........ try again" << endl;
    }
    else
        for (size_t i = 0; i < menuItems.size(); i++) {
            if (ord == menuItems[i]->getItemNo()) {
                int total = quantity * menuItems[i]->getPrice();
                cout << "Total: " << total << endl;
                cout << "*************** THANKS FOR PREFERING ***************"
                break; // Exit after finding the item
            }
        }
}
admin::~admin(void) {
    for (size_t i = 0; i < menuItems.size(); i++) {
```

```
        delete menuItems[i]; // Free dynamically allocated memory
    }
}
void admin::createAndSaveOrderHistory(const string& obj, const vector <string>&
    ofstream file(obj); // Open the file for writing

    if (!file.is_open()) {
        cerr << "Error opening file: " << obj << endl;
        return;
    }

    for (size_t i = 0; i < orders.size(); ++i) {
        file << "Order " << (i + 1) << ": " << orders[i] << endl;
    }

    file.close();
}

void admin::readOrderHistory(const string& filename) {
    ifstream file(filename); // Open the file for reading
    string line;

    if (!file.is_open()) {
        cerr << "Error opening file: " << filename << endl;
        return;
    }

    cout << "Order History:" << endl;

    while (getline(file, line)) {
        cout << line << endl;
    }
```

```
    file.close();
}

void admin::placeOrder(const string& order) {
    orderHistory.push_back(order); // Add order to the history
}
void admin::notify() {
    cout << " your order will be ready after 15 minutes : please note down your
}
/*so fstream is used to open /close read and write the file , so there are thre
fstream is a base class
if stream is derived from  base class
ofstream is used to derived it from base class
there are two types to open this file ie
constructor of that class
and member function open()*/
```

5.2.3 Source.cpp Main file

```
   #include "MenuItemBase.h"
#include <fstream>
#include <iostream>
// For file operations
#include <string>
using namespace std;
int main() {
 cout << "************ WELCOME TO WILDGOOSE CAFE**" << endl;
 admin  myCafe;

 myCafe.add(1, "chicken burger", 350);
 myCafe.add(2, "zinger burger", 500);
 myCafe.add(3, "potatochips", 200);
 myCafe.add(4, "lassania", 900);
 myCafe.add(5, "chocolate souffle", 750);
```

```
myCafe.add(6, "apple pie", 520);
myCafe.add(7, " zinger strips", 200);
myCafe.add(8, " coffee", 200);
myCafe.add(9, " tea", 100);
myCafe.add(10, "juice", 70);
myCafe.add(11, "Coffee", 50);
myCafe.add(12, "Tea", 30);
myCafe.add(13, "Sandwich", 70);

myCafe.order();



   string name;
   int itemnumber;
   int quantity;

   // Step 1: Create and write to the file
   ofstream outFile("Text.txt"); // Open file for writing
   if (!outFile) {
       cerr << "Error opening file for writing!" << endl;
       return 1;
   }

   cout << "Enter student records (Enter 'done' to stop):\n";
   while (true) {
       cout << "Enter name (or 'done' to stop): ";
       cin >> name;
       if (name == "done") break;

       cout << "Enter item number: ";
       cin >> itemnumber;

       cout << "Enter quantity: ";
```

```
        cin >> quantity;

        // Write data to the file
        outFile << name << " " << itemnumber << " " << quantity << endl;
    }
    outFile.close(); // Close the file

    // Step 2: Read from the file
    ifstream inFile("Text.txt"); // Open file for reading
    if (!inFile) {
        cerr << "Error opening file for reading!" << endl;
        return 1;
    }

    cout << "\norder Records from ' Text.txt':\n";
    while (inFile >> name >> itemnumber >> quantity) { // Read until EOF
        cout << "Name: " << name << ", Item number: " << itemnumber << ",quanti
    }
    inFile.close(); // Close the file



 myCafe.notify();

 return 0;
}
```